# SOFTWARE DESIGN DOCUMENT

*CampusConnect: University Club, Event & Activity Management Platform*

| Attribute | Specification Detail |
|---|---|
| Document Identifier | CC-SDD-2026-V1.0 (IEEE Std 1016-2009 Standard) |
| Engineering Version | Version 1.0 (Architecture Baseline) |
| Software Engineering Team | Md. Shakhawat Hossain (Student ID: 0432320005101009) \| Fahim Hossain (Student ID: 0432320005101007) \| Department of CSE |
| Course Advisor & Supervisor | Al Imtiaz \| Department of Computer Science and Engineering |
| Reviewing Body | Faculty Academic Review Board \| Division of Computing and Systems |
| Release Date & Status | October 2026 \| Approved for Engineering Implementation |

## 1. Introduction and Architectural Goals

### 1.1 Purpose

This Software Design Document (SDD) provides the definitive technical architecture and low-level system design for CampusConnect, a centralized university extracurricular and club management platform. This specification bridges the high-level functional requirements documented in the Software Requirements Specification (CC-SRS-2026-V2.0) and the implementation codebase. It details the modular component decomposition, concurrency mechanics, data architecture, RESTful API contracts, and cryptographic verification pipelines.

### 1.2 System Scope

CampusConnect consolidates student organizational activities across university departments. The platform orchestrates 4 distinct stakeholder roles: Students, Club Executives, Faculty Advisors, and System Administrators. Primary design objectives include:

- **1. ACID Transactional Integrity:** Absolute prevention of event over-registration and race conditions during high-volume registration rushes via row-level locking.
- **2. Anti-Proxy Attendance Assurance:** Elimination of attendance fraud (such as QR screenshot sharing among students) through dynamic, short-lived rotating cryptographic tokens.
- **3. Verifiable Digital Credentials:** Generation of cryptographically signed, tamper-proof completion certificates accessible via a public validation endpoint.
- **4. Deterministic Auditability:** Comprehensive, immutable audit trails capturing every lifecycle change, approval, rejection, and role elevation.

- **5. Responsive Performance:** API latency targets with p95 < 250ms for read operations and p95 < 400ms for transactional writes under concurrent loads.

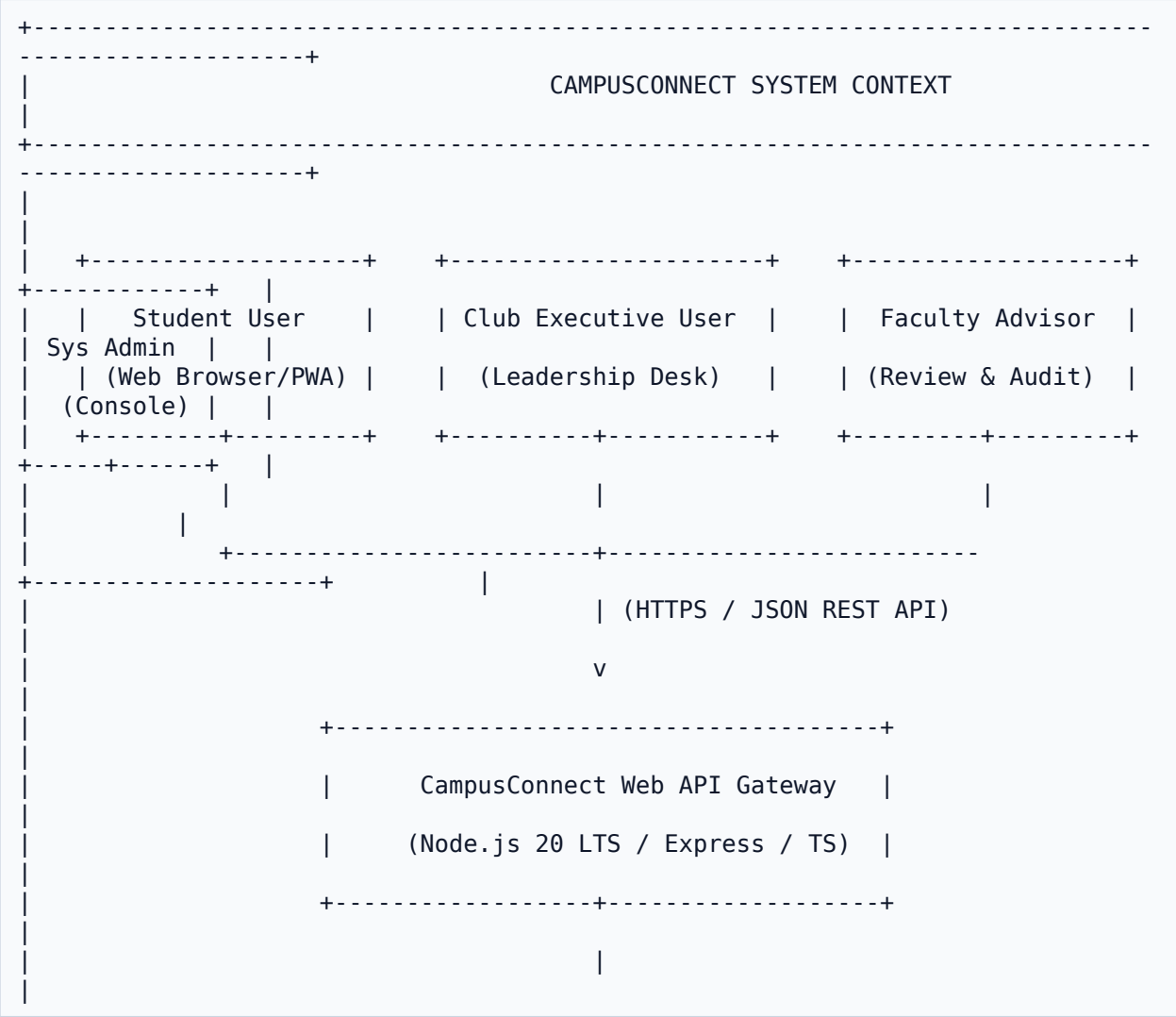
### 1.3 Design Constraints and Standards

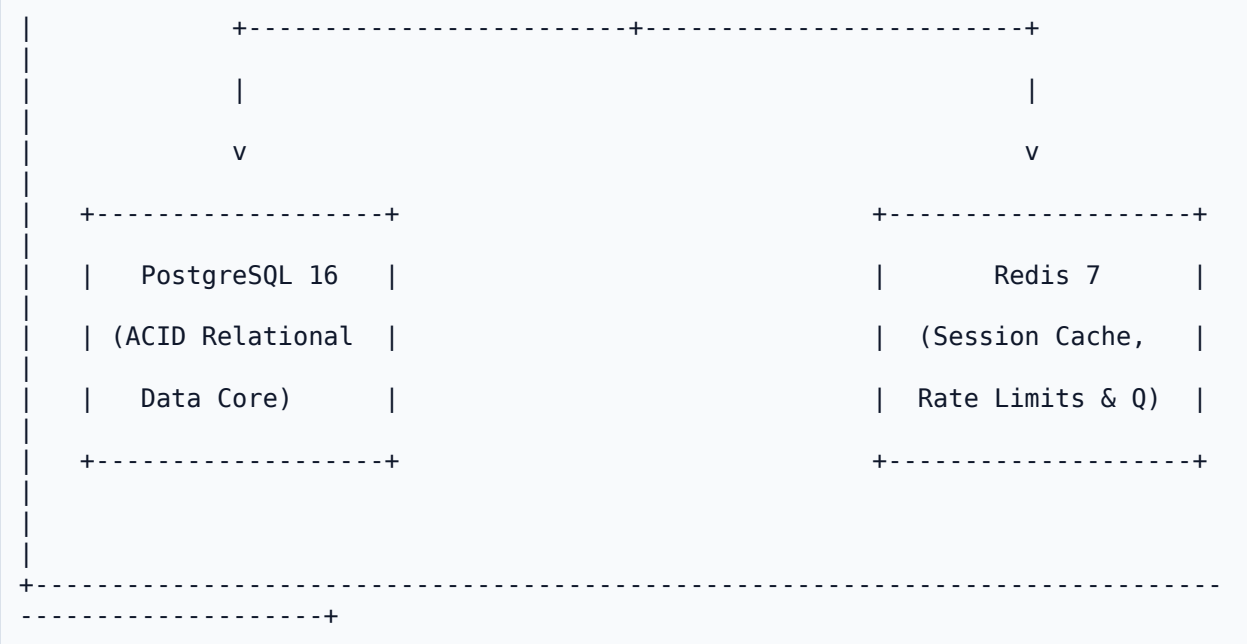
The system design strictly adheres to IEEE Std 1016-2009 for software design documentation, RFC 7519 for JWT token handling, RFC 6238 for time-based one-time password (TOTP) generation, and RFC 7807 for REST API problem details. Furthermore, all documents and source code strictly avoid typographic em-dashes and en-dashes, employing ASCII hyphens, colons, and pipes exclusively.

## 2. Architectural Overview and System Context

### 2.1 C4 Context Level Architecture

CampusConnect operates within the campus cloud network, integrating four primary human actors and background workers into a coherent, secure topology:






### 2.2 C4 Container Level Decomposition

1. Client Web Application: Single-page application built with TypeScript, communicating with backend REST endpoints via secure HTTP-only cookies and JSON payloads.

2. REST API Gateway: Node.js 20+ LTS runtime executing Express.js with TypeScript. Enforces input validation schemas (Zod), JWT verification, and RBAC authorization barriers.

3. Relational Core: PostgreSQL 16 server housing all persistent normalized entities with ACID guarantees, foreign keys, and row-level locking.

4. Fast Cache & Queue Engine: Redis 7 providing sub-millisecond session state lookup, token blacklist validation, and token-bucket rate limiting.

5. Asynchronous Job Worker: Dedicated worker processing PDF certificate rendering and notification dispatches without blocking event loop workers.

## 3. Architectural Decision Records (ADRs)

### 3.1 ADR-01: Modular Monolith Architecture

Decision: Build CampusConnect as a modular monolith in Node.js/TypeScript rather than distributed microservices.

Rationale: Microservices introduce distributed transactions, eventual consistency hazards, network serialization latency, and deployment complexity. A modular monolith provides clean domain boundary separation while preserving single-database ACID transactions essential for slot reservations.

### 3.2 ADR-02: PostgreSQL 16 Relational Engine

Decision: Mandate PostgreSQL 16 as the persistent data store, rejecting document-based NoSQL engines.

Rationale: CampusConnect data is inherently relational: users belong to clubs, clubs own events, events receive registrations, registrations yield attendance, and attendance earns certificates. PostgreSQL offers battle-tested ACID compliance, foreign key constraints, composite indexing, and row-level locking (SELECT ... FOR UPDATE) required to eliminate race conditions during registration bursts.

### 3.3 ADR-03: Dynamic HMAC/TOTP Attendance Tokens

Decision: Implement dynamic rotating QR codes with 15 to 30 second validity windows.

Rationale: Standard static QR codes displayed on paper or static screens can be easily photographed, sent via instant messaging, and scanned by absent peers, compromising attendance validity. Rotating cryptographic tokens ensure that only physically present attendees viewing the live display or presenting a refreshed personal token can be checked in.

### 3.4 ADR-04: Cryptographically Verifiable Digital Certificates

Decision: Issue PDF certificates containing embedded SHA-256 digests and public verification URLs.

Rationale: Event completion certificates are frequently presented on student resumes and scholarship portfolios, creating a risk of digital forgery. Third-party evaluators, employers, and academic departments can verify authenticity instantly without needing system credentials.

### 3.5 ADR-05: Secure Password Hashing with bcrypt (and Argon2id Roadmap)

Decision: Enforce secure password hashing using bcrypt with 12 salt rounds across all student and faculty credentials, with an architectural migration abstraction in place for Argon2id.

Rationale: bcrypt is an industry-standard, battle-tested cryptographic hash providing tunable computational cost against brute-force attacks. The system encapsulates hashing within the authentication service to enable seamless migration to Argon2id in a future major release.

## 4. Component and Module Design

### 4.1 Authentication and Role-Based Access Control (RBAC)

The platform strictly segments privileges into 4 access tiers. Every incoming request traverses the authentication middleware to verify JWT signatures, followed by the authorization barrier checking specific role permissions.

| Resource Entity | Operation | Student | Club Exec | Advisor | Sys Admin |
|---|---|---|---|---|---|
| User Profile | View / Edit Self | Yes | Yes | Yes | Yes |
| User Accounts | Assign System Roles | No | No | No | Yes |
| Clubs | Browse Directory | Yes | Yes | Yes | Yes |
| Clubs | Create / Disable Club | No | No | No | Yes |
| Clubs | Edit Metadata | No | Yes (Own) | Yes (Own) | Yes |
| Club Members | Request Membership | Yes | No | No | No |
| Club Members | Approve / Remove | No | Yes (Own) | Yes (Own) | Yes |
| Events | View Published | Yes | Yes | Yes | Yes |
| Events | Draft & Submit | No | Yes (Own) | No | Yes |
| Events | Approve / Reject | No | No | Yes (Assigned) | Yes |
| Registrations | Register / Cancel Self | Yes | Yes | Yes | Yes |
| Registrations | View Full Roster | No | Yes (Own) | Yes (Assigned) | Yes |
| Attendance | Present Dynamic QR | Yes | Yes | Yes | Yes |
| Attendance | Scan & Check-in | No | Yes (Own) | Yes (Assigned) | Yes |
| Certificates | Download Self | Yes | Yes | Yes | Yes |
| Certificates | Verify Public Hash | Public | Public | Public | Public |
| Audit Logs | Query Audit Trail | No | No | No | Yes |

## 4.2 Concurrency-Safe Event Registration Engine

To ensure that event registrations never exceed established room or venue capacities during sudden registration spikes, all slot allocations are encapsulated within transactional row-level locking:

```
BEGIN;
-- Lock the event row against concurrent modifications
SELECT id, capacity, registered_count, status, registration_deadline
FROM events WHERE id = $event_id FOR UPDATE;
```

```
-- Verify registration validity
IF status != 'PUBLISHED' OR NOW() > registration_deadline THEN
    ROLLBACK; RAISE EXCEPTION 'Registration closed';
END IF;

-- Verify user registration existence
IF EXISTS(SELECT 1 FROM registrations WHERE user_id = $user_id AND event_id =
$event_id) THEN
    ROLLBACK; RAISE EXCEPTION 'User already registered';
END IF;

-- Allocate confirmed slot or enqueue onto waitlist in registrations table
IF registered_count < capacity THEN
    INSERT INTO registrations (id, event_id, user_id, ticket_code, status,
waitlist_position, created_at)
    VALUES (gen_random_uuid(), $event_id, $user_id, $ticket_code, 'CONFIRMED',
NULL, NOW());
    UPDATE events SET registered_count = registered_count + 1 WHERE id =
$event_id;
    COMMIT; -- Confirmed slot allocated
ELSE
    SELECT COALESCE(MAX(waitlist_position), 0) + 1 INTO next_pos
    FROM registrations WHERE event_id = $event_id AND status = 'WAITLISTED';
    INSERT INTO registrations (id, event_id, user_id, ticket_code, status,
waitlist_position, created_at)
    VALUES (gen_random_uuid(), $event_id, $user_id, $ticket_code, 'WAITLISTED',
next_pos, NOW());
    COMMIT; -- Placed into FIFO waitlist position
END IF;
```

### 4.3 Dynamic Anti-Proxy Attendance Mechanism

The attendance module prevents proxy check-ins by generating rotating HMAC tokens refreshed every 20 seconds. The server accepts tokens from the current time step (T0) and provides a 20-second grace window (T-1) to accommodate minor network latency. Screenshots shared with absent peers expire before they can be transmitted and submitted.

### 4.4 Verifiable Digital Certificate Pipeline

Upon event completion and attendance validation, eligible students can download cryptographically signed completion certificates. Each certificate features an immutable serial (CC-2026-UUID8), a SHA-256 integrity hash calculated over student details and event metadata, and a QR link pointing directly to the public verification endpoint.

## 5. Relational Data Architecture and Schema

The persistent storage layer is structured into 13 normalized relational tables in PostgreSQL 16:

| Table Name | Primary Key | Foreign Key Dependencies | Critical Indexing |
|---|---|---|---|
| users | id (UUID) | None | UNIQUE(email) |
| student_profiles | id (UUID) | user_id -> users(id) | UNIQUE(user_id), UNIQUE(student_id_number) |
| event_categories | id (UUID) | None | UNIQUE(name) |
| clubs | id (UUID) | faculty_advisor_id -> users(id) | UNIQUE(slug), UNIQUE(name) |
| club_memberships | id (UUID) | club_id -> clubs, user_id -> users | UNIQUE(club_id, user_id) |
| events | id (UUID) | club_id -> clubs, created_by -> users | INDEX(club_id), INDEX(status) |
| registrations | id (UUID) | event_id -> events, user_id -> users | UNIQUE(event_id, user_id), UNIQUE(ticket_code) |
| attendance_records | id (UUID) | event_id -> events, user_id -> users, reg_id -> registrations | UNIQUE(registration_id), INDEX(event_id, user_id) |
| certificates | id (UUID) | event_id -> events, user_id -> users | UNIQUE(serial_number), UNIQUE(event_id, user_id) |
| announcements | id (UUID) | club_id -> clubs, author_id -> users | INDEX(club_id) |
| notifications | id (UUID) | user_id -> users | INDEX(user_id, is_read) |
| feedback | id (UUID) | event_id -> events, user_id -> users | UNIQUE(event_id, user_id) |
| audit_logs | id (UUID) | actor_id -> users(id) | INDEX(timestamp DESC) |

## 6. RESTful API Contracts and Interfaces

The backend exposes structured RESTful JSON endpoints adhering to RFC 7807 problem details for all client error responses:

| Method | Route Endpoint | Access | Payload / Action Description |
|---|---|---|---|
| POST | /api/v1/auth/register | Public | Register student account with university email and student profile. |

| Method | Route Endpoint | Access | Payload / Action Description |
|---|---|---|---|
| POST | /api/v1/auth/login | Public | Authenticate credentials; return JWT & set refresh cookie. |
| POST | /api/v1/auth/refresh | Cookie | Exchange refresh token for refreshed 15-minute JWT. |
| POST | /api/v1/auth/logout | Auth | Revoke refresh token and invalidate client cookies. |
| GET | /api/v1/auth/me | Auth | Retrieve profile and permissions of authenticated user. |
| GET | /api/v1/auth/users | Sys Admin | List all registered accounts with role filters. |
| PATCH | /api/v1/auth/users/:id/role | Sys Admin | Update user system role assignment. |
| GET | /api/v1/clubs | Public | List clubs with category and active status filtering. |
| GET | /api/v1/clubs/:idOrSlug | Public | Retrieve full club profile, executive board, and events. |
| POST | /api/v1/clubs | Sys Admin | Create a newly chartered student organization. |
| POST | /api/v1/clubs/:clubId/apply | Student | Submit membership request to target club. |
| GET | /api/v1/clubs/:clubId/ members | Auth | Retrieve member roster with status filtering. |
| PATCH | /api/v1/clubs/:clubId/ members/:membershipId | Club Exec | Approve or reject pending membership request. |
| GET | /api/v1/clubs/:clubId/ analytics | Club Exec | Retrieve executive activity and membership metrics. |
| GET | /api/v1/events | Public | Browse upcoming published events with category filter. |
| GET | /api/v1/events/:id | Public | Retrieve event details, remaining slots, and speaker info. |
| POST | /api/v1/events | Club Exec | Create new event proposal in DRAFT state. |

| Method | Route Endpoint | Access | Payload / Action Description |
|---|---|---|---|
| POST | /api/v1/events/:id/submit | Club Exec | Submit draft event to faculty advisor for review. |
| POST | /api/v1/events/:id/review | Advisor | Approve or reject event with mandatory written notes. |
| POST | /api/v1/events/:eventId/register | Student | Atomic slot registration or waitlist reservation. |
| POST | /api/v1/registrations/:registrationId/cancel | Student | Cancel registration; triggers automated waitlist promotion. |
| GET | /api/v1/events/:eventId/attendance/live-qr | Club Exec | Retrieve rotating HMAC attendance token for live display. |
| POST | /api/v1/events/:eventId/attendance/scan | Student | Submit dynamic QR token for attendance verification. |
| POST | /api/v1/events/:eventId/attendance/manual | Club Exec | Organizer fallback check-in for manual validation. |
| POST | /api/v1/events/:eventId/certificates/issue | Club Exec | Generate tamper-proof completion certificates. |
| GET | /api/v1/certificates/me | Student | List all certificates earned by authenticated student. |
| GET | /api/v1/certificates/verify/:serialNumber | Public | Stateless public verification of certificate validity. |
| GET | /api/v1/analytics/admin | Sys Admin | Query system-wide metrics and institutional KPIs. |
| GET | /api/v1/health | Public | Service liveness and health probe. |

# 7. Security and Non-Functional Architecture

## 7.1 Threat Mitigation Strategy

| Vulnerability Vector | Targeted Asset | CampusConnect Defense Mechanism |
|---|---|---|
| SQL Injection | Relational Database | Parameterized queries with native PostgreSQL bindings. |

| Vulnerability Vector | Targeted Asset | CampusConnect Defense Mechanism |
|---|---|---|
| Cross-Site Scripting (XSS) | Client Web App | HTML escaping and strict Content Security Policy (CSP). |
| Cross-Site Request Forgery | Authenticated APIs | SameSite=Strict cookie policy and CORS origin lock. |
| Race Conditions | Event Capacity | Pessimistic row-level locking (SELECT ... FOR UPDATE). |
| Credential Stuffing | Auth Endpoints | Redis token-bucket rate limiting (5 attempts per 15 min). |
| Certificate Forgery | Completion Credentials | Cryptographic SHA-256 signature and public verification. |
| Proxy Attendance | Check-in QR Pass | Dynamic TOTP tokens with 20s rotation and short TTL. |

## 7.2 Performance and Latency Budgets

1. Read Latency: p95 response time < 250ms for club and event discovery under 500 concurrent connections.

2. Transactional Write Latency: p95 response time < 400ms for event registration under 200 concurrent requests.

3. Asynchronous Worker Throughput: Certificate generation completed within 60 seconds of event closing.

4. Availability Standard: 99.5% uptime during active academic semesters with automated health checks.